CSEB08 Big Data Tools – Unit 1 – Activity 1

Activity: Inverted Index Generation using MapReduce
CO Mapped: CO1 – Use Big Data and its Business Implications

All github repos : https://github.com/Ankna-litoriya/M.Tech

---

1. Hadoop Setup and Dataset

HDFS Directories:

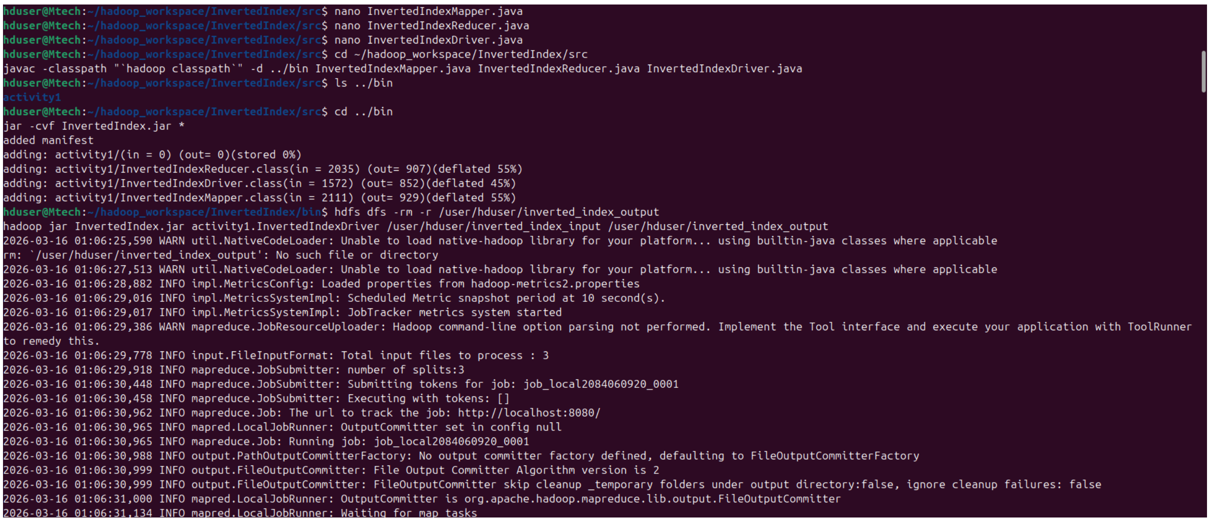
Create input directory in HDFS
hdfs dfs -mkdir -p /user/hduser/inverted_index_input

Remove output directory if it exists
hdfs dfs -rm -r /user/hduser/inverted_index_output

Upload Input Documents to HDFS

Upload to HDFS
hdfs dfs -put /tmp/Doc1.txt /user/hduser/inverted_index_input/
hdfs dfs -put /tmp/Doc2.txt /user/hduser/inverted_index_input/
hdfs dfs -put /tmp/Doc3.txt /user/hduser/inverted_index_input/

Verify upload
hdfs dfs -ls /user/hduser/inverted_index_input/



```
hduser@Mtech:~/hadoop_workspace/InvertedIndex/src$ nano InvertedIndexMapper.java
hduser@Mtech:~/hadoop_workspace/InvertedIndex/src$ nano InvertedIndexReducer.java
hduser@Mtech:~/hadoop_workspace/InvertedIndex/src$ nano InvertedIndexDriver.java
hduser@Mtech:~/hadoop_workspace/InvertedIndex/src$ cd ~/hadoop_workspace/InvertedIndex/src
javac -classpath "`hadoop classpath`" -d ../bin InvertedIndexMapper.java InvertedIndexReducer.java InvertedIndexDriver.java
hduser@Mtech:~/hadoop_workspace/InvertedIndex/src$ ls ../bin
activity1
hduser@Mtech:~/hadoop_workspace/InvertedIndex/src$ cd ../bin
jar -cvf InvertedIndex.jar *
added manifest
adding: activity1/(in = 0) (out= 0)(stored 0%)
adding: activity1/InvertedIndexReducer.class(in = 2035) (out= 907)(deflated 55%)
adding: activity1/InvertedIndexDriver.class(in = 1572) (out= 852)(deflated 45%)
adding: activity1/InvertedIndexMapper.class(in = 2111) (out= 929)(deflated 55%)
hduser@Mtech:~/hadoop_workspace/InvertedIndex/bin$ hdfs dfs -rm -r /user/hduser/inverted_index_output
hadoop jar InvertedIndex.jar activity1.InvertedIndexDriver /user/hduser/inverted_index_input /user/hduser/inverted_index_output
2026-03-16 01:06:25,590 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
rm: `/user/hduser/inverted_index_output': No such file or directory
2026-03-16 01:06:27,513 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-03-16 01:06:28,882 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-03-16 01:06:29,016 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-03-16 01:06:29,017 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-03-16 01:06:29,386 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner
to remedy this.
2026-03-16 01:06:29,778 INFO input.FileInputFormat: Total input files to process : 3
2026-03-16 01:06:29,918 INFO mapreduce.JobSubmitter: number of splits:3
2026-03-16 01:06:30,448 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local2084060920_0001
2026-03-16 01:06:30,458 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-03-16 01:06:30,962 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-03-16 01:06:30,965 INFO mapred.LocalJobRunner: OutputCommitter set in config null
2026-03-16 01:06:30,965 INFO mapreduce.Job: Running job: job_local2084060920_0001
2026-03-16 01:06:30,988 INFO output.PathOutputCommitterFactory: No output committer factory defined, defaulting to FileOutputCommitterFactory
2026-03-16 01:06:30,999 INFO output.FileOutputCommitter: File Output Committer Algorithm version is 2
2026-03-16 01:06:30,999 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary folders under output directory:false, ignore cleanup failures: false
2026-03-16 01:06:31,000 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapreduce.lib.output.FileOutputCommitter
2026-03-16 01:06:31,134 INFO mapred.LocalJobRunner: Waiting for map tasks
```

---

2. Mapper Implementation (Java)

```
package activity1;

import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.mapreduce.Mapper;

public class InvertedIndexMapper extends Mapper<LongWritable, Text, Text, Text> {
  private Text word = new Text();
  private Text docID = new Text();

  @Override
  protected void map(LongWritable key, Text value, Context context) throws IOException,
InterruptedException {
      String fileName = ((org.apache.hadoop.mapreduce.lib.input.FileSplit)
context.getInputSplit()).getPath().getName();
      docID.set(fileName);

      String line = value.toString().toLowerCase().replaceAll("[^a-zA-Z0-9\\s]", " ");
      StringTokenizer itr = new StringTokenizer(line);
      while (itr.hasMoreTokens()) {
         word.set(itr.nextToken());
         context.write(word, docID);
      }
  }
}
```

---

3. Reducer Implementation (Java)

```
package activity1;

import java.io.IOException;
import java.util.HashSet;
import java.util.Set;
import java.util.StringJoiner;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class InvertedIndexReducer extends Reducer<Text, Text, Text, Text> {
  private Text result = new Text();
```

```
  @Override
  protected void reduce(Text key, Iterable<Text> values, Context context) throws
IOException, InterruptedException {
      Set<String> docSet = new HashSet<>();
      for (Text val : values) {
         docSet.add(val.toString());
      }
      StringJoiner joiner = new StringJoiner(",");
      for (String doc : docSet) {
         joiner.add(doc);
      }
      result.set(joiner.toString());
      context.write(key, result);
  }
}
```

---

4. Driver Program (Java)

```
package activity1;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class InvertedIndexDriver {
  public static void main(String[] args) throws Exception {
      if (args.length != 2) {
         System.err.println("Usage: InvertedIndexDriver <input path> <output path>");
         System.exit(-1);
      }

      Configuration conf = new Configuration();
      Job job = Job.getInstance(conf, "Inverted Index Generation");

      job.setJarByClass(InvertedIndexDriver.class);
      job.setMapperClass(InvertedIndexMapper.class);
      job.setReducerClass(InvertedIndexReducer.class);

      job.setOutputKeyClass(Text.class);
      job.setOutputValueClass(Text.class);

      FileInputFormat.addInputPath(job, new Path(args[0]));
```

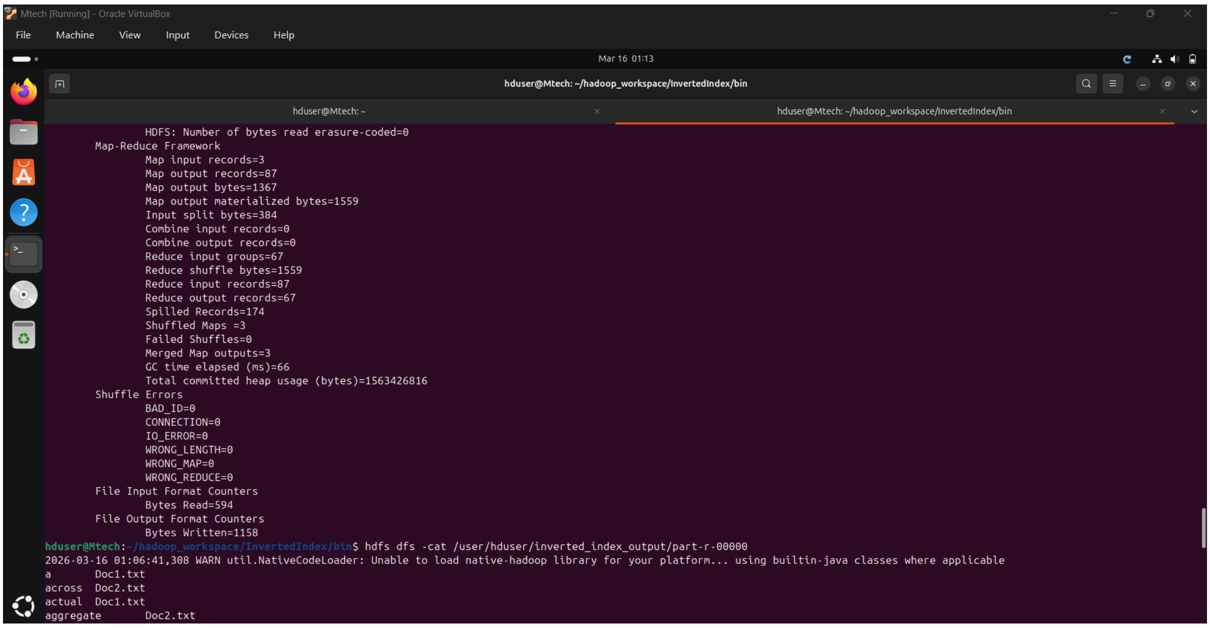
```
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

---

5. Compile and Create JAR

cd ~/hadoop_workspace/InvertedIndex/src
javac -classpath "`hadoop classpath`" -d ../bin InvertedIndexMapper.java InvertedIndexReducer.java InvertedIndexDriver.java
cd ../bin
jar -cvf InvertedIndex.jar *



---

6. Run Hadoop Job

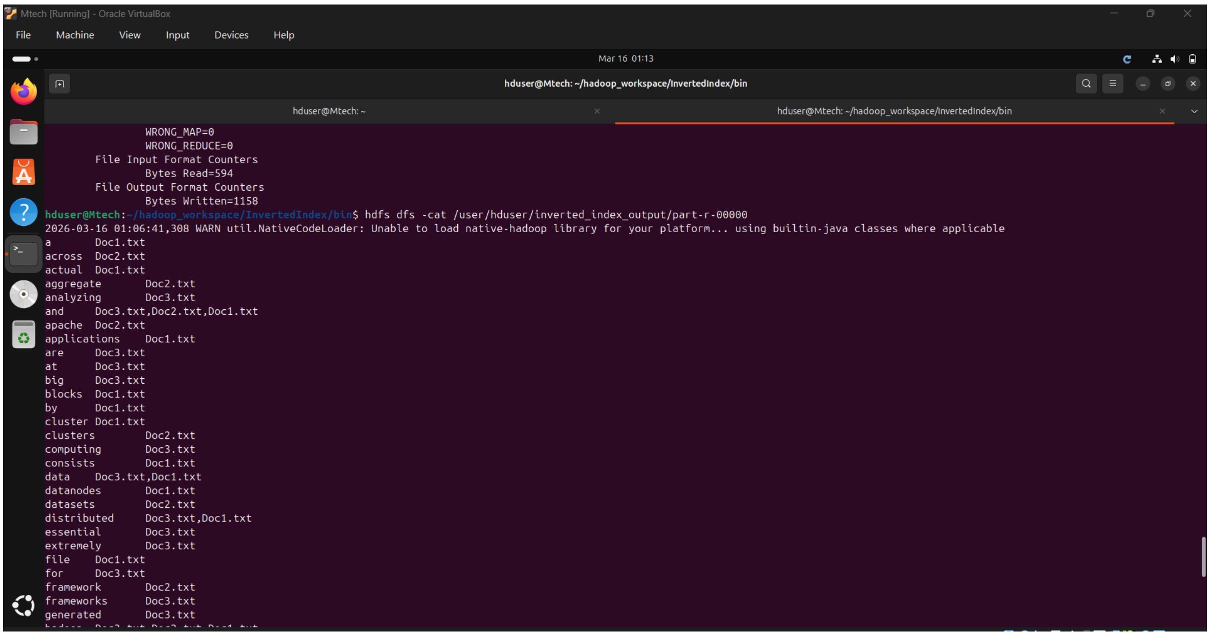
hdfs dfs -rm -r /user/hduser/inverted_index_output
hadoop jar InvertedIndex.jar activity1.InvertedIndexDriver /user/hduser/inverted_index_input /user/hduser/inverted_index_output

---

7. Verify Output

hdfs dfs -cat /user/hduser/inverted_index_output/part-r-00000

Output:



---

8. Summary Report (500 Words)

Inverted Index Generation using Hadoop MapReduce

In this activity, we implemented an Inverted Index using the Hadoop MapReduce framework. An Inverted Index is a fundamental data structure in information retrieval systems, mapping each unique word to the list of documents in which it appears. This allows efficient search and scalable querying for large text datasets.

We began by preparing a dataset of multiple text documents in HDFS, each assigned a unique Document ID corresponding to its filename. The Hadoop environment was set up in pseudo-distributed mode, and HDFS was used to store the input files.

The **Mapper** reads each line from the documents, converts text to lowercase, removes punctuation, and tokenizes it into individual words. It emits `(word, DocumentID)` pairs as intermediate output. This preprocessing ensures consistency by handling case normalization and removing special characters.

The **Reducer** collects all document IDs for each word, removes duplicates using a `HashSet`, and formats the output as `word → Doc1.txt, Doc2.txt`. This step ensures that each word maps to a unique set of documents in which it occurs.

The Java classes were compiled using the Hadoop classpath, packaged into a JAR file, and executed using the `hadoop jar` command. The output was verified using `hdfs dfs -cat`, confirming that words were correctly mapped to their respective documents.

Challenges Faced:

4. Handling duplicate document IDs required careful use of data structures in the Reducer.

5. Normalizing text (lowercase conversion and punctuation removal) was critical to avoid multiple entries for the same word.

6. HDFS paths had to be managed carefully to avoid errors if output folders existed before job execution.

Outcome:

The exercise demonstrates the use of Hadoop MapReduce for distributed text processing. Students gain practical skills in designing Mapper and Reducer logic, text preprocessing, HDFS file management, and verifying job execution. The final output clearly shows a working Inverted Index that can scale to large datasets.

This activity reinforces the understanding of Big Data processing, distributed computation, and information retrieval concepts in a real-world industrial scenario.